

Tema: Programación de microcontroladores: Creación de drivers GPIO en STM32CubeIDE.

Tipo de Actividad de Formación Práctica: Laboratorio

Unidad Temática 6.- Microprocesadores.

Trabajo de Laboratorio: Creación de drivers GPIO, desde cero, en STM32CubeIDE.

1. Alcance

El objetivo de este trabajo de laboratorio es:

- Implementar un driver para el módulo GPIO utilizando STM32CubeIDE. funcionamiento en placa de desarrollo STM32.
- Integrar el driver en una aplicación previamente desarrollada.
- Evaluar su funcionamiento en placa de desarrollo STM32.

2. Elementos / Instrumental a Utilizar

- Placa de desarrollo STM32-NUCLEO-F4XX (modelos sugeridos: F401-RE, F412-ZG, F413-ZH, F429-ZI).
- Computadora con software de desarrollo integrado STM32CubeIDE instalado.
- Repositorio grupal en GitHub para integrar los resultados.
- Documentación de apoyo: 'Creación de driver GPIO.pdf'.
- Material de referencia incluido en el IDE (STM32CubeIDE Manuals).

3. Actividades

- a- Seleccionar una aplicación básica previamente desarrollada que maneje encendido de Leds como base de trabajo.
- b- Crear un driver para el módulo GPIO siguiendo la guía de creación de drivers.
- c- Modificar el archivo main.c para reemplazar llamadas a la HAL por las funciones del driver creado.
- d- Compilar, depurar y programar la placa STM32 verificando el correcto funcionamiento.
- e- Replicar la modificación en todas las aplicaciones de la AFP1, asegurando coherencia en nombres de drivers y funciones.
- f- Subir las aplicaciones al repositorio grupal en GitHub en la carpeta correspondiente.
- g- Elaborar un informe en formato PDF que contenga: introducción, descripción de las aplicaciones modificadas, observaciones, y link al repositorio

4. Duración: 3 Hs

Desarrollo

Introducción:

En el desarrollo de sistemas embebidos, los *drivers* cumplen un rol fundamental al actuar como la capa de software encargada de permitir la interacción segura y eficiente entre el hardware y las aplicaciones de más alto nivel. En plataformas basadas en microcontroladores STM32, la creación y modificación de drivers es un proceso esencial para personalizar el comportamiento de los periféricos, optimizar el

rendimiento del sistema y garantizar un control adecuado de los recursos disponibles.

Dentro del entorno STM32CubeIDE, los drivers se estructuran siguiendo la arquitectura HAL (Hardware Abstraction Layer), que proporciona funciones estandarizadas para la configuración y operación de periféricos como GPIO, temporizadores, ADC, UART, SPI e I2C. Esta abstracción facilita el desarrollo, al mismo tiempo que permite al programador realizar modificaciones específicas cuando se requieren funcionalidades avanzadas o comportamientos no contemplados por las librerías predeterminadas.

El presente informe describe las aplicaciones modificadas durante el desarrollo del trabajo práctico, detallando los cambios realizados en los drivers, su motivación técnica y las mejoras alcanzadas. Además, se incluyen observaciones relevantes del proceso de implementación y validación, junto con el enlace al repositorio donde se encuentra el código desarrollado.

Descripcion de las modificaciones en cada aplicacion:

Aplicacion 1.1:

La primera aplicación desarrollada implementa una secuencia de encendido y apagado de tres LEDs de la placa, ejecutando un arrastre donde cada LED se activa por un breve intervalo antes de pasar al siguiente. En la versión original del trabajo práctico, esta funcionalidad se encontraba programada directamente dentro del main.c mediante el uso de funciones HAL, lo que generaba un código acoplado, difícil de mantener y poco reutilizable.

Para mejorar la estructura y cumplir con los objetivos de modularización, se incorporó un *driver* propio de manejo de GPIO, ubicado en la carpeta Drivers/API. Este driver encapsula todas las operaciones relacionadas con la inicialización de pines y control de los LEDs, ofreciendo funciones de más alto nivel como writeLedOn_GPIO(), writeLedOff_GPIO(),

`toggleLed_GPIO()` y `delay_ms()`. Gracias a esta capa de abstracción, el archivo `main.c` ya no depende directamente de HAL, sino que utiliza una interfaz independiente del hardware.

En el programa principal, los pines correspondientes a LD1, LD2 y LD3 se almacenan en un vector, lo que permite recorrerlos secuencialmente dentro del lazo principal. Mediante el uso de la función `toggleLed_GPIO()` y retardos de 200 ms, cada LED se enciende y apaga de manera ordenada, generando la animación de arrastre requerida por la consigna. Esta reorganización mejora la claridad del código, facilita la portabilidad de la aplicación y permite reutilizar el driver en futuras prácticas.

Observaciones:

Durante la implementación se observó que la utilización del driver de GPIO permitió una separación clara entre la lógica de la aplicación y las funciones específicas del hardware. El archivo `main.c` quedó reducido únicamente a la lógica de control del arrastre, mientras que todos los detalles de inicialización de pines y manejo de puertos quedaron encapsulados dentro del módulo `API_GPIO`. Esto mejora la portabilidad del código, ya que ante un cambio de placa o de pines solamente sería necesario modificar el driver, manteniendo intacta la lógica de la aplicación.

Asimismo, el uso de un vector para almacenar los pines de los LEDs permitió evitar código repetitivo y facilitó la expansión a un número distinto de LEDs con solo modificar la constante `CANT_LEDS`. La aplicación funcionó correctamente, cumpliendo con el patrón de encendido secuencial solicitado en la actividad.

Aplicacion 1.2:

La segunda aplicación implementa un sistema de control de luces basado en dos secuencias de encendido de LEDs, utilizando un driver modular para el manejo del GPIO. A diferencia de una implementación directa en el main.c, esta aplicación separa la lógica de hardware en un driver propio (API_GPIO), lo cual mejora la legibilidad, la escalabilidad y la portabilidad del código.

En esta aplicación, los tres LEDs disponibles en la placa Nucleo se organizan mediante un vector (vector_leds) y se controlan a través de dos funciones principales: secuencia1() y secuencia2(). Cada secuencia define un patrón distinto de encendido y apagado, con un retardo configurable mediante la constante delay_ms.

La variable sec_actual actúa como una máquina de estados simple, alternando entre las dos secuencias. El cambio de secuencia se realiza mediante la lectura del pulsador de usuario (USER_Btn). El driver detecta el estado del botón mediante la función readButton_GPIO(), la cual se utiliza dentro de las secuencias junto con un antirrebote por software.

Observaciones:

- Se utiliza una **máquina de estados simple**, lo cual es una buena práctica para aplicaciones secuenciales.
- El driver abstrae correctamente los accesos a pines del microcontrolador.
- Se incluye antirrebote por software dentro de las secuencias, lo cual evita falsos cambios.
- Los delays están implementados dentro de bucles internos de las funciones de secuencia, lo cual mantiene la simplicidad aunque limita la posibilidad de multitarea.

- El uso de un vector de LEDs permite extender la aplicación fácilmente a más salidas.

Aplicacion 1.3:

Reemplazo total de funciones HAL por funciones del driver GPIO.

Toda operación que antes se hacía con:

- HAL_GPIO_WritePin()
- HAL_GPIO_TogglePin()
- HAL_GPIO_ReadPin()
- HAL_Delay()

ahora se realiza exclusivamente mediante las funciones de la API:

- writeLedOn_GPIO()
- writeLedOff_GPIO()
- toggleLed_GPIO()
- readButton_GPIO()
- delay_ms()

Con esto, la aplicación deja de depender directamente del HAL.

La App incorpora la lectura del pulsador a través del driver.
para detectar presiones y resolver los cambios de secuencia.

Antes el botón se leía directamente con HAL; ahora la lectura queda encapsulada en el driver.

Se implementa un vector de LEDs utilizando definiciones del driver.

El driver define:

```
#define CANT_LEDS 3  
typedef uint16_t led_t;
```

La App utiliza estas definiciones para construir:

```
uint16_t vector_leds[CANT_LEDS];
```

evitando así valores hardcodeados y delegando en el driver la configuración del hardware.

Observaciones:

- Buena separación entre capa de aplicación y capa de drivers.
La App solo ejecuta lógica de secuencias y lectura de botón; el manejo del hardware queda centralizado en el driver.
- Mayor portabilidad del código.
Si se cambia de microcontrolador o de librería HAL, solo se debería modificar el driver, no la aplicación.
- Código más ordenado y mantenible.
La abstracción evita repetición de configuraciones y permite que las secuencias se escriban de forma clara y reutilizable.
- Uso correcto de typedef y defines en la API.
Simplifica la administración del número de LEDs y hace que la aplicación sea independiente del hardware específico.

Aplicacion 1.4:

En esta cuarta aplicación se incorporaron los drivers de la API_GPIO, por lo que la estructura original basada en acceso directo al hardware mediante HAL fue reemplazada por funciones encapsuladas. Las principales modificaciones fueron:

Uso de vectores de configuración

- Se definió un vector `vector_leds` con los pines de los 3 LEDs.

- Se agregó un vector `vector_tiempos` con los diferentes tiempos de parpadeo.
- Se incorporó la variable `caso` para manejar una máquina de estados simple que cambia entre los distintos tiempos.

Reemplazo del manejo directo de LEDs por funciones del driver

Antes:

```
HAL_GPIO_WritePin(GPIOB, LD1_Pin, GPIO_PIN_SET);
```

Ahora:

```
writeLedOn_GPIO(vector_leds[j]);
```

Se reemplazaron todas las llamadas HAL por:

- `writeLedOn_GPIO()`
- `writeLedOff_GPIO()`
- `toggleLed_GPIO()`

Esto permite desacoplar la aplicación del hardware.

Inlcusion del driver del boton

Antes:

```
HAL_GPIO_ReadPin(GPIOC, USER_Btn_Pin)
```

Ahora:

```
readButton_GPIO()
```

Dentro de la función `secuencia()` se usa el driver para:

- Leer el botón
- Aplicar antirrebote por software
- Cambiar la velocidad de parpadeo (ciclo entre 100–1000 ms)

Observaciones:

Buen uso de la abstracción

La app queda correctamente desacoplada del hardware. Si mañana se cambia la placa o los pines, solo se modifica API_GPIO.c y no la aplicación.

La función secuencia() es una mejora real

- Ordena la lógica
- Facilita el mantenimiento
- Permite la reutilización en otros proyectos

La máquina de estados es sencilla pero efectiva

caso recorre secuencias de tiempo:

100 → 250 → 500 → 1000 → vuelve a 100

Muy clara y bien aplicada.

Se respeta el objetivo del laboratorio

La aplicación demuestra:

- Uso de drivers propios
- Separación aplicación/hardware
- Encapsulación de funciones
- Manejo de GPIO mediante APIs

Aplicaciones desarrolladas

App 2.1: https://github.com/Julihubgit/Grupo-3-TDII-2025/blob/main/AFP_3_TDII_2025/AFP_2_GRUPO_3_App.1.1.zip

App 2.2: https://github.com/Julihubgit/Grupo-3-TDII-2025/blob/main/AFP_3_TDII_2025/AFP_2_GRUPO_3_App.1.2.zip

Universidad Tecnológica Nacional – Facultad Regional Tucuman
Carrera: Ingenieria Electronica Asignatura: Tecnicas Digitales II
Actividad de Formacion Practica N°3 - LAB - Año 2025

App 2.3: https://github.com/Julihubgit/Grupo-3-TDII-2025/blob/main/AFP_3_TDII_2025/AFP_2_GRUPO_3_App.1.3.zip

App 2.4: https://github.com/Julihubgit/Grupo-3-TDII-2025/blob/main/AFP_3_TDII_2025/AFP_2_GRUPO_3_App.1.4.zip

Universidad Tecnológica Nacional – Facultad Regional Tucuman

Carrera: Ingenieria Electronica Asignatura: Tecnicas Digitales II

Actividad de Formacion Practica N°3 - LAB - Año 2025